

CSE2430 - Operating Systems

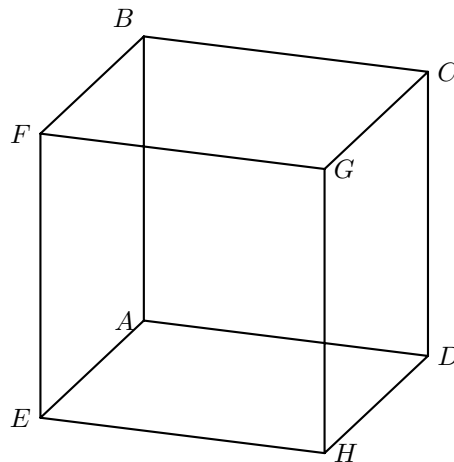
Homework Synchronization

Turn-in Instructions

No hand-in required. Questions are meant for self-directed exam preparation and can be discussed in the midterm Q&A session.

Question 1

Consider a 3D variant of the Dining Philosophers problem. In this problem, there are 8 philosophers that are located on the vertices of a 3D cube. Each philosopher needs to grab 3 forks, which are located on the edges it is connected to, and are identified by the vertices they connect. For example, in the illustration below, philosopher F needs to grab forks $fork_{F,B}$, $fork_{F,G}$ and $fork_{F,E}$ to be able to eat. Each fork is protected by a non-blocking lock, whose API specifies that a lock attempt is either successful (and the lock is then acquired), or unsuccessful (and then the lock is not acquired). Explain how one could design a solution such that philosophers can eat forever without deadlock and without starvation issues.



Question 2

Name one advantage and one disadvantage of using processes for concurrency instead of threads.

Question 3

Explain why a generic spin lock is not fair and what makes a ticket lock fair.

Question 4

The `test_and_set` atomic instruction sets a variable *var* to a new value *new_value* and returns its old value *old_value*:

old_value = `test_and_set(&var, new_value)`.

Explain how to implement the two operations of a lock using `test_and_set`.

Question 5

What happens in Java when you call `wait()` on an object outside a synchronized block involving this object? What would happen if the runtime system would behave differently?